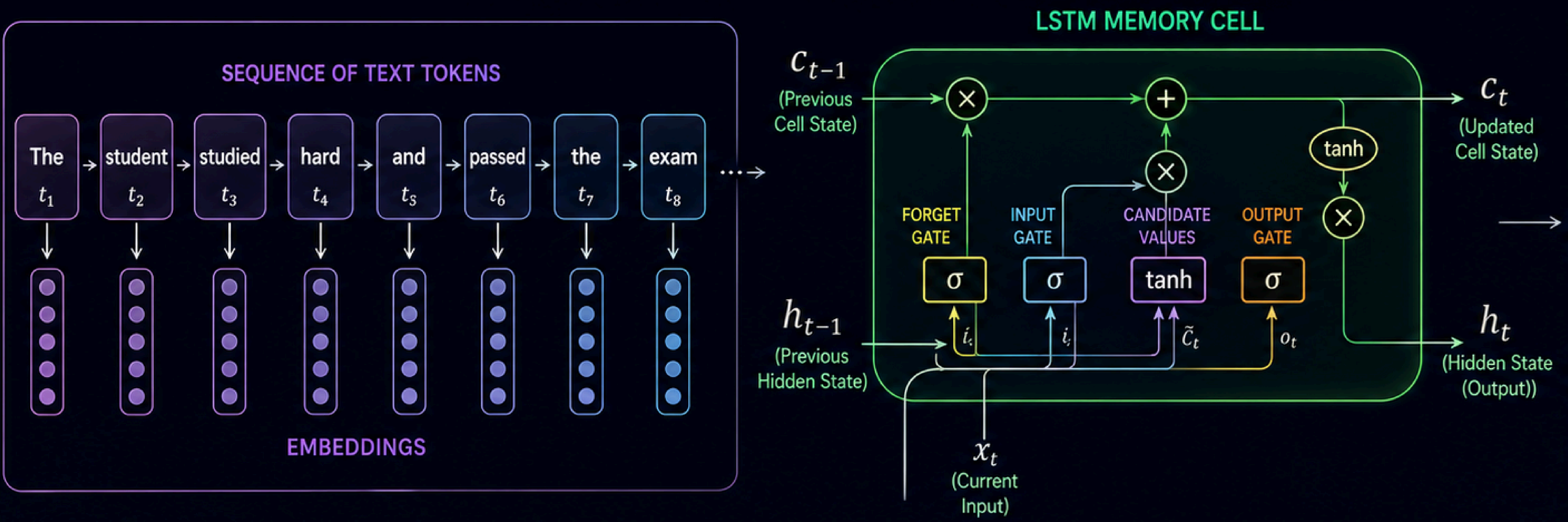
Mastering Sequence and Context

LSTMs and Natural Language Processing



USAGE

The final hidden state captures the context and meaning of the entire sequence.

Used for tasks like:

- Sentiment Analysis
- Text Generation
- Machine Translation
- Question Answering

And more...

Language is sequential. Meaning depends on what came before.

LSTMs remember what matters and forget what doesn't.

They learn long-term dependencies in data with context.

Better context = smarter models = better results.

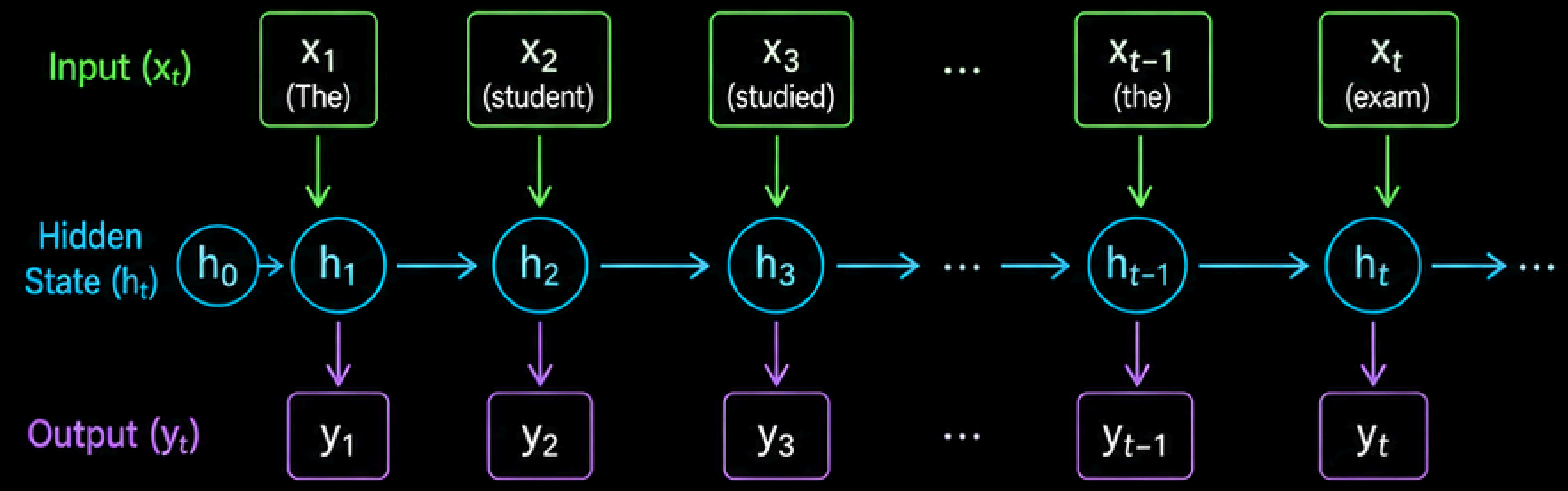
The Problem with Standard RNNs: Forgetting the Past

1 Standard Recurrent Neural Networks (RNNs) suffer from the **Vanishing Gradient Problem.**

2 During backpropagation through time, **gradients shrink exponentially.**

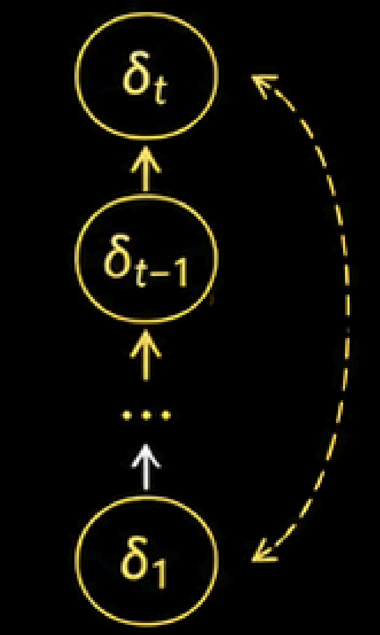
3 Result: The network **"forgets"** the beginning of a long paragraph by the time it reaches the end.

STANDARD RNN UNFOLDED OVER TIME



BACKPROPAGATION THROUGH TIME (BPTT)

Error at time t



Gradients are multiplied by Jacobians at each time step.

Repeated multiplication **by values < 1** causes gradients to vanish.

THE FORGETTING EFFECT

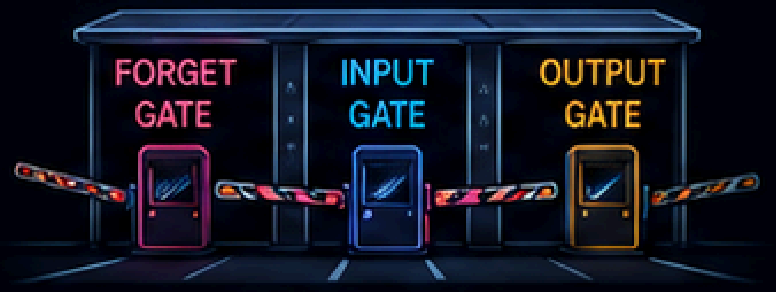
By the time the network reaches the end, information from the beginning is almost lost.

KEY TAKEAWAY: Standard RNNs struggle with long-range dependencies. → They can't retain early information effectively. → We need a better memory mechanism. Enter **LSTMs!**

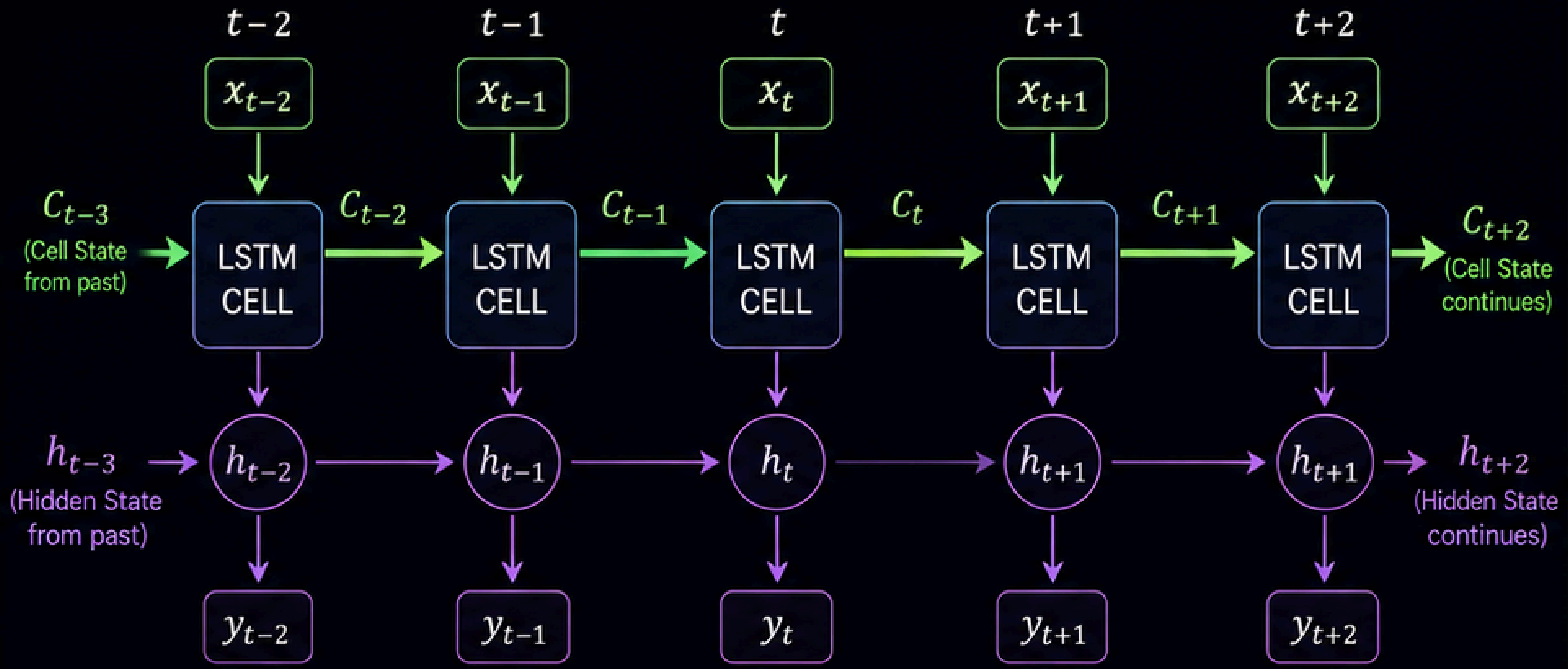
Enter the LSTM: The Long Memory Highway

1 LSTMs solve this by introducing the Cell State—a direct mathematical highway running straight down the entire chain with minimal interference.

2 Information is carefully added or removed from this highway using specialized neural layers called Gates.



THE LSTM UNFOLDED OVER TIME



==== Cell State (C): The long-term memory highway. Information flows with minimal changes.

→ Hidden State (h): The short-term memory and output of the LSTM cell at each time step.

THE THREE GATES

Control the information flow on the highway.

FORGET GATE (f_t)
Decides what information to remove from the cell state.
Think: "What should I forget?"

INPUT GATE (i_t)
Decides what new information to add to the cell state.
Think: "What should I remember?"

OUTPUT GATE (o_t)
Decides what part of the cell state to output as the hidden state.
Think: "What should I output?"

KEY TAKEAWAY:

The Cell State is a "memory highway" that preserves information across long distances.

→ Gates act like smart toll booths that regulate what gets on, what gets off, and what gets through.

→ This design allows LSTMs to capture long-term dependencies effectively.

The Tools of the Trade (Part 3)

The Three Gates of an LSTM: Regulating the Flow

1 FORGET GATE

Decides what information from the past to throw away using a Sigmoid layer (0 = forget completely, 1 = keep completely).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

SIGMOID OUTPUT (f_t)

INTERPRETATION

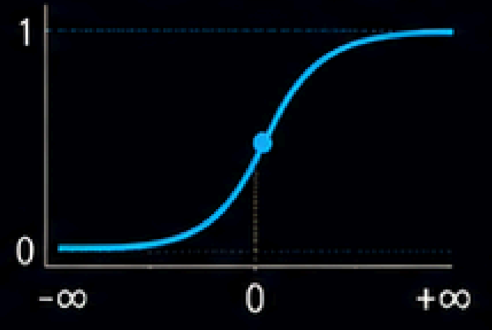
- 0 = Forget completely (remove from cell state)
- 0.5 = Partial forget (scale down)
- 1 = Keep completely (preserve in cell state)

2 INPUT GATE

Decides what new information to store in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

SIGMOID OUTPUT (i_t)



CANDIDATE VALUES ($\tilde{c}_t$)

New candidate information (created with a tanh layer)

$$\tilde{c}_t = tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

INTERPRETATION

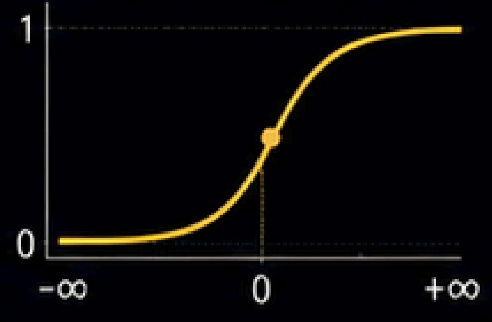
The input gate decides how much of the new candidate information should be added to the cell state.

3 OUTPUT GATE

Decides what the next hidden state should be, based on a filtered version of the cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

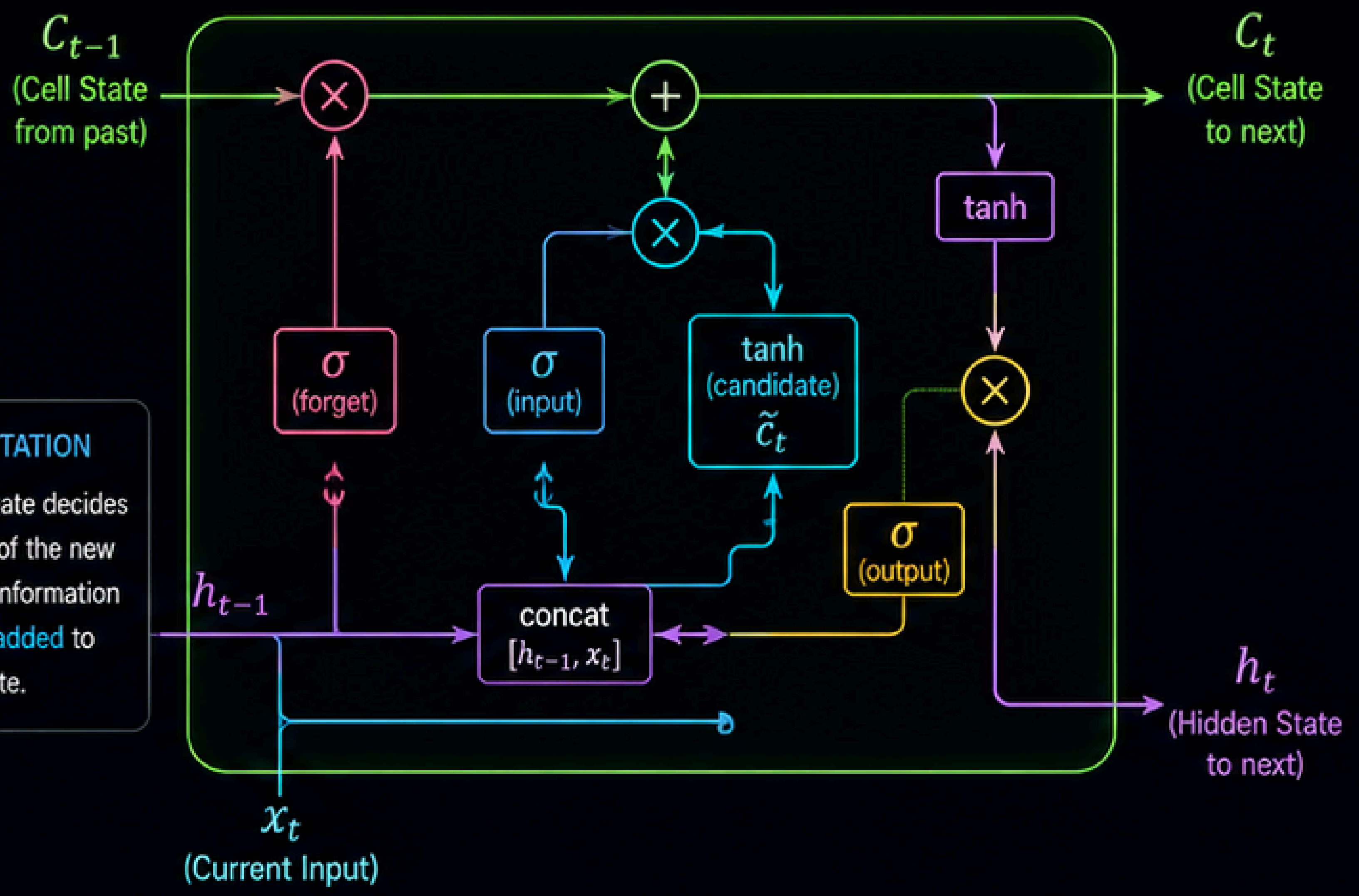
SIGMOID OUTPUT (o_t)



INTERPRETATION

The output gate controls how much of the (filtered) cell state is exposed as the next hidden state h_t.

LSTM CELL AT TIME t



LEGEND

- σ — Sigmoid (0 to 1)
- tanh — Tanh (-1 to 1)
- ⊗ — Element-wise Multiplication
- ⊕ — Element-wise Addition
- concat [...] — Concatenation (Join vectors)

KEY TAKEAWAY: The forget gate cleans the memory. + The input gate writes new information. = The output gate decides what to reveal. → Together, they regulate the flow of information through time.

The Tools of the Trade (Part 4)

Natural Language Processing (NLP) Prep: Math Cannot Read Words

1 TOKENIZATION

Breaking sentences down into individual words or sub-words.

Example Sentence

The quick, brown fox jumps over the lazy dog.

Tokenized Output (Words)

The	quick	,	brown	fox	jumps	over	the	lazy	dog	.

OR

Tokenized Output (Sub-words using WordPiece Example)

The	quick	,	brown	fox	jump	##s	over	the	lazy	dog	.

2 VOCABULARY INDEXING

Assigning a unique integer to every token.
(e.g., “The” = 1, “Parser” = 2)

Vocabulary (Example)

The →	1
Parser →	2
quick →	3
brown →	4
fox →	5
... →	...
dog →	V

Tokenized Sentence

The	quick	,	brown	fox	jumps	over	the	lazy	dog	.

Indexed Sequence

1	3	,	4	5	6	7	8	3	9	10	11

(Numbers correspond to the vocabulary index table on the left)

3 PADDING

Neural networks require uniform input shapes. We pad shorter sentences with zeros so all inputs match the longest sequence.

Example Sentences / **After Tokenization + Indexing + Padding (max length = 6)**

Example Sentence	Length	Sequence							
The cat sat on the mat	(len = 6)	Seq 1	1	12	34	56	23	78	← no padding needed
NLP is fascinating	(len = 3)	Seq 2	79	14	80	0	0	0	← padded with zeros
Transformers are powerful models	(len = 4)	Seq 3	81	82	83	84	0	0	← padded with zeros

All sequences are now the same length (6), ready for the neural network!

KEY TAKEAWAY:

Text is broken into tokens. → Tokens are mapped to integers. → Sequences are padded to the same length. → Now the math can begin!

The Tools of the Trade (Part 5)

Word Embeddings: Capturing Semantic Meaning

1 Passing raw integers to an LSTM is inefficient.

1 | 12 | 34 | 56 | 23 | 78 → LSTM → Inefficient

Word Indices

FROM TEXT TO EMBEDDINGS

Input Sentence: The parser checks the syntax of the code.

Tokenization:

The	parser	checks	the	syntax	of	the	code	.
1	57	213	4	621	6	4	99	2

→ Embedding Layer → LSTM → ...

Dense vectors (e.g., 100 dimensions)

2 Embedding Layers map each word into a dense, high-dimensional vector space (e.g., 100 dimensions).

Word Indices: 1 | 12 | 34 | 56 | 23 | 78 → Embedding Layer →

Dense Vectors (100 Dimensions)

0.12	-0.35	0.47	...	0.21
-0.08	0.71	-0.23	...	0.05
0.33	-0.12	0.61	...	-0.31
⋮	⋮	⋮	...	⋮
0.19	0.42	-0.07	...	0.66

100 Dimensions

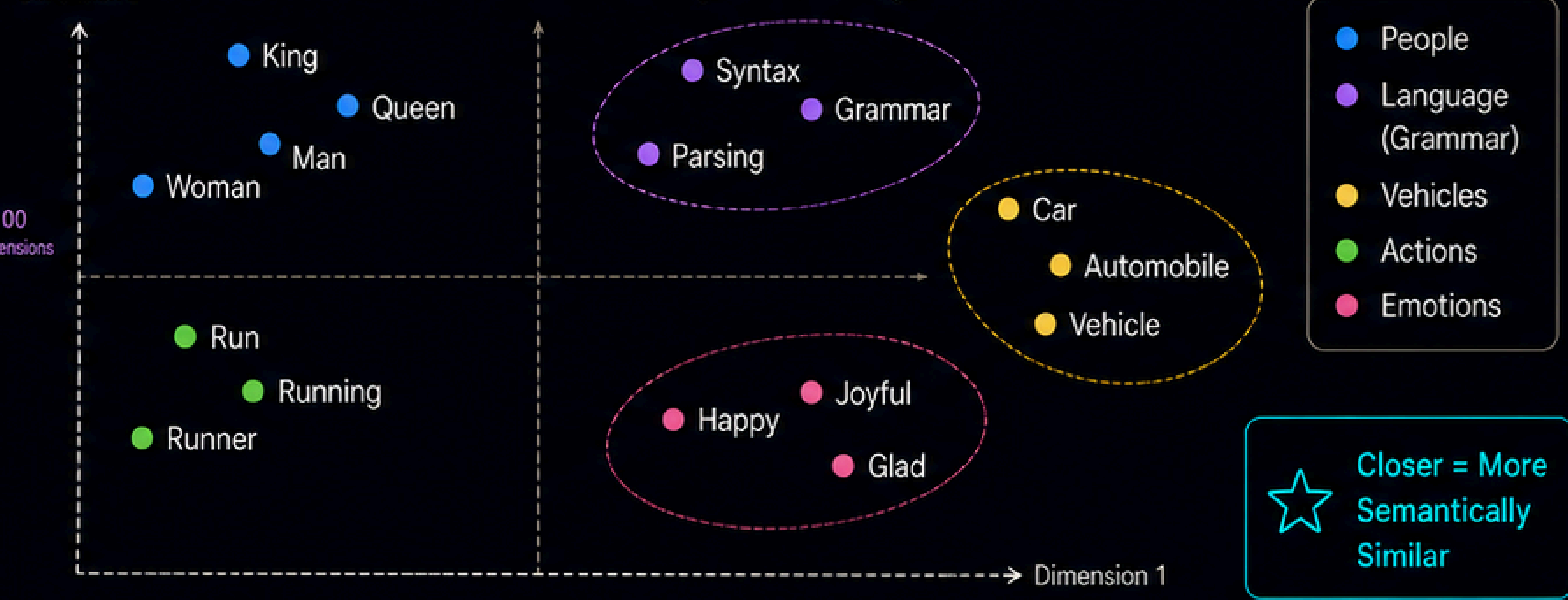
(One dense vector per word)

3 In this space, words with similar meanings (like "Syntax" and "Grammar") are mathematically closer together.

Embeddings capture semantic relationships between words.

SEMANTIC VECTOR SPACE (2D PROJECTION)

Words with similar meanings are closer together.



Closer = More Semantically Similar

KEY TAKEAWAY:

1 | 12 | 34 — Integers alone lack semantic meaning. → Embeddings convert words into meaningful vectors. → The vector space captures relationships between words. → This helps LSTMs understand context and meaning better.

The Tools of the Trade (Part 6)

NLP Applications: What Can LSTMs Do?

1 SEQUENCE CLASSIFICATION

Sentiment analysis or categorizing research papers.

Goal:
Assign a single label to the entire sequence.

How it works

Input Sentence

I | love | this | movie | !

... → LSTM → LSTM → LSTM → LSTM → LSTM → ... → h_T → Dense (Softmax) →

Last Hidden State (h_T)

Output (Sentiment)

Positive	0.92
Neutral	0.06
Negative	0.02

Example

Input:
I love this movie!

Predicted Label:
Positive (92%)

2 SEQUENCE-TO-SEQUENCE

Machine translation (e.g., translating English to Tamil).

Goal:
Convert one sequence into another sequence.

How it works

Encoder (English)

How | are | you | ?

... → LSTM → LSTM → LSTM → LSTM → Context Vector (c)

Decoder (Tamil)

<SOS> → LSTM → LSTM → LSTM → LSTM → LSTM → ...

நான் | நலம் | தானே | ? | <EOS>

Example

Input (English):
How are you?

Output (Tamil):
நான் நலம் தானே ?

3 TEXT SUMMARIZATION

Condensing long documents into 1-2 human-like sentences.

Goal:
Generate a shorter, meaningful summary.

How it works

Long Document

Artificial intelligence (AI) has rapidly advanced in recent years. It is being used in healthcare, finance, education, and many other domains. Machine learning, a subset of AI, enables systems to learn from data. Deep learning, a part of machine learning, uses neural networks with many layers...

→ LSTM Encoder-Decoder →

Generated Summary

AI is transforming multiple domains, including healthcare, finance, and education. Machine learning and deep learning are key drivers behind these advances.

Example

Input:
Long research paper / article

Output Summary:
AI is transforming multiple domains through machine learning and deep learning.

KEY TAKEAWAY:

LSTMs understand the order and context of words. → This enables powerful NLP applications across many real-world tasks. → From understanding sentiment to translating languages, LSTMs bring text to life.